

Where Does the Time Go? Cold Starts of On-Demand Node.js Web Servers on a Personal Machine

Dhanush Gowdhaman
University of Pennsylvania
dhanushg@seas.upenn.edu

Abstract—Scale-to-zero is no longer only a cloud feature. Developers run gateways on their own laptops that start a web server when its domain is first requested and stop it when idle, trading memory for a cold start on the next visit. We ask where that cold start goes for real Node.js servers and what a gateway can do about it *without changing the applications*. With a preloaded probe we measure 15 servers, from a bare `node:http` server to Next.js, NestJS and self-hosted applications, on an Apple M4 Pro under Node.js 25. Cold starts take 45–576 ms (median 87 ms): a fixed 35 ms of runtime, then initialization of which the module system accounts for a median 72%, growing with the number of modules loaded. For small servers the launcher costs more than the server: 150 ms readiness polling adds 66 ms, and pm2 adds 32 ms, almost all of it one default that injects a monitoring agent into every application. Of six application-transparent techniques, a pool of pre-started runtimes helps most (geometric-mean latency factor 0.62) and bundling helps where it is correct (11 of 15 servers); V8’s compile cache barely helps (0.98), and user-land startup snapshots are infeasible for every server because `node:http` cannot be serialized. Freezing an idle server instead of stopping it resumes in 2.3 ms but keeps all of its memory. Applying the findings, including removing avoidable round trips from the gateway’s own control path, cuts an on-demand gateway’s cold-start latency by a median 45% (29–59%).

Index Terms—cold start, Node.js, scale-to-zero, process startup, module loading, V8, personal computing

I. INTRODUCTION

Serverless platforms made *scale-to-zero* familiar: an idle function holds no resources, and the first request after a pause pays a *cold start* [1], [2]. The same idea has moved onto personal machines. Tools such as puma-dev [3] and Sablier [4] start a local service when its hostname is requested and stop it after a period of inactivity, and developers who self-host several small web applications on one laptop build similar gateways around process managers such as pm2 [5]. The motivation is the same as in the cloud (idle servers hold memory, and a laptop has little to spare), but the setting is not: the servers are ordinary, unmodified web applications rather than functions; there is one user, whose every cold start is felt directly; and there are no containers, microVMs, or checkpoint/restore facilities to lean on.

Almost all cold-start research targets the cloud platform: sandbox creation, container and VM snapshots, and keep-alive policy [6]–[12]. Application-level work reduces code loading for serverless *functions* [13]–[15], mostly in Python. We found

no systematic study of where the cold start of a complete Node.js web server goes on a single machine, of how much the launcher itself adds, or of whether the startup features that recent Node.js releases added—an on-disk V8 compile cache [16] and user-land startup snapshots [17]—help real servers.

We built an on-demand gateway (LOCALSERVER) for our own applications, noticed that its cold starts were dominated by things other than the applications, and set out to measure. This paper contributes:

- A **measurement method** for Node.js server cold starts: a preloaded probe that records Node’s startup milestones, module loading, the moment the server listens, and a sampled CPU profile of initialization, with no changes to the application.
- A **characterization of 15 servers** (framework baselines, self-hosted applications, and the personal applications that motivated this work), decomposing each cold start into process launch, runtime boot, module loading and initialization, and the first request.
- A **decomposition of launcher overhead**: process manager, readiness detection, socket handoff, and the gateway’s own control path.
- An **evaluation of six application-transparent techniques**—compile cache, bundling, startup snapshots, a pre-started runtime pool, socket pre-binding, and freezing—including where they cannot be applied at all and why.
- An **end-to-end result** in a working gateway, and the harness, corpus and data as an artifact.

II. SETTING AND BACKGROUND

A. On-demand hosting on a personal machine

Fig. 1 shows the path of the first request to a stopped service. A gateway listens on the machine’s HTTP port and routes by hostname (`time.localhost`, `project.local`). When a request arrives for a service that is not running, the gateway asks a *launcher* to start it, waits until the server is *ready*, and then forwards the request. The gateway stops the service after an idle timeout. In LOCALSERVER, the launcher is pm2 [5], used because it provides logging, restarts and visibility for services the user also manages by hand, and readiness is detected by probing the service’s TCP port every 150 ms—a common default.

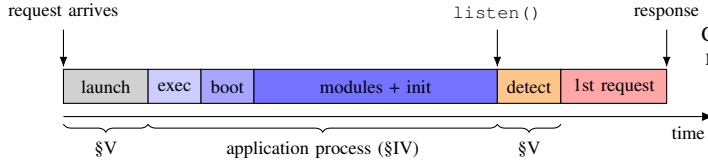


Fig. 1. Anatomy of a cold start. The application process (blue) is the subject of §IV; the launcher and readiness detection (grey, orange) of §V. Widths are illustrative.

B. How a Node.js server starts

Starting `node server.js` creates a process, initializes V8 and the Node.js environment (largely from a built-in startup snapshot), runs any `-require/-import` preloads, and then loads the entry module. Loading a module means resolving its specifier to a file, reading it, compiling it, linking it (ES modules), and evaluating it, which loads its own imports. Only after the whole import graph has been evaluated and any initialization (opening databases, reading configuration) has run does the server call `listen()`. Two recent runtime features target this path. The *module compile cache* (Node.js 22.1+) persists V8 code caches to disk, so later runs can skip compilation [16]. *User-land startup snapshots* (Node.js 18.8+) serialize a heap after running an initialization script, so later runs can deserialize it instead of re-running initialization [17], [18].

III. METHODOLOGY

A. Corpus

Table I lists the 15 servers. *Framework* baselines are minimal “hello” servers that differ only in framework: `node:http`, Express 5, Fastify 5, Koa 3, Hono 4, NestJS 11, and a Next.js 15 production server rendering one dynamic page. *Real* servers are self-hosted applications installed from npm with local storage: `json-server`, the Verdaccio npm registry, and Actual Budget’s sync server. *User* servers are the five personal applications that motivated this work, run from copies with copies of their data: a time tracker (Express, JSON files), a project tracker and a job tracker (plain `node:http` with `node:sqlite`), a pm2 web dashboard (Fastify), and the time tracker’s Vite development server. Eight use ES modules as their entry; the corpus loads between ≈ 1 and 1,065 modules before listening.

B. Instrumentation

A probe (≈ 200 lines) is loaded with `node -require` before the application. It reads Node’s startup milestones (`performance.nodeTiming`), wraps the CommonJS loader to time top-level loads, wraps `net.Server.prototype.listen` to report the instant the application’s server starts listening, and reports over an inherited pipe. On request it also samples a CPU profile (100 μ s interval) between the first user code and `listen()`, which we categorize by call frame into the module system (`node:internal/modules`), other Node internals, dependency code (`node_modules`), application code,

TABLE I
CORPUS. MODULES: MODULES LOADED BEFORE `listen()`. COLD START: MEDIAN SPAWN-TO-FIRST-RESPONSE WITH A 95% BOOTSTRAP CI, IDEAL LAUNCHER (§IV).

App	group	entry	modules	cold start (ms)	bundle
node-http	framework	CJS	1	45 [44, 45]	✓
express	framework	CJS	125	70 [69, 71]	✓
fastify	framework	CJS	245	87 [87, 88]	✓
koa	framework	CJS	55	67 [67, 68]	✓
hono	framework	ESM	28	60 [60, 61]	✓
nextjs	framework	CJS	735	135 [133, 136]	✓
nextjs	framework	CJS	242	237 [236, 240]	–
json-server	real	ESM	85	75 [74, 76]	✓
verdaccio	real	CJS	1024	277 [276, 278]	✓
actual-server	real	ESM	1065	576 [570, 580]	–
time-backend	user	ESM	132	76 [75, 76]	✓
project-tracker	user	ESM	5	58 [55, 59]	–
job-tracker	user	ESM	233	100 [99, 102]	✓
pm2-dashboard	user	ESM	523	152 [150, 153]	✓
vite-dev	user	ESM	16	132 [131, 133]	–

V8/native work (which includes parsing and compilation), garbage collection, and idle time. Profiling and module counting run in separate passes from timing, because both perturb what they measure.

For each run the harness spawns the server on a fresh port and timestamps the spawn; the probe’s `listen` report triggers the first request, followed by five warm requests. All timestamps share one epoch clock (`performance.timeOrigin`). We decompose each run into *process launch* (spawn to Node’s time origin), *runtime boot* (to the first user code), *modules + init* (to `listen()`), and the *first request* (to its complete response).

C. Experiments

We report five experiments: **E1** decomposes cold starts under an ideal launcher (§IV); **E2** adds launchers and readiness detection, and **E2b** isolates pm2’s monitoring agent (§V); **E3** evaluates application-transparent techniques and **E4** freezing (§VI); and **E5** measures a working gateway end to end (§VII).

D. Environment and statistics

All measurements ran on an Apple M4 Pro (12 cores, 24 GB) with macOS 26.6 and Node.js 25.2.0, on AC power, Low Power Mode off, and otherwise idle. Each configuration of each server was run 30 (E1) or 20 (E2, E3) times after a discarded warm-up, with configurations interleaved in random order per repetition so that drift affects all equally. The page cache was warm, as it is for a service a user restarts; we could not flush it without privileges and discuss this under threats to validity. We report medians with 95% bootstrap confidence intervals, and call a difference clear only when the bootstrap interval of the difference in medians excludes zero.

IV. WHERE DOES THE TIME GO?

Fig. 2 decomposes each server’s cold start under an *ideal launcher*: the harness spawns the server directly and learns the instant it listens, so nothing outside the server is counted. Cold starts range from 45 to 576 ms with a median of 87 ms.

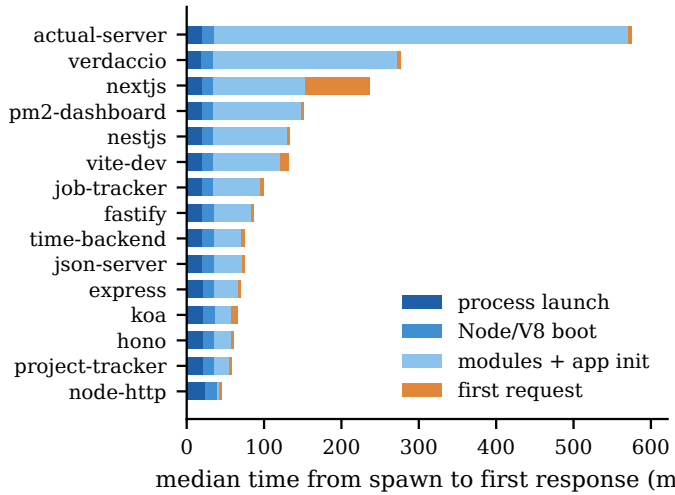


Fig. 2. Where a cold start goes, per server (medians of 30 runs, ideal launcher).

Repetitions are tight: the coefficient of variation is at most 12%.

Finding 1. Node.js costs a fixed 35 ms before any application code runs.

Process launch (spawn to Node’s time origin) takes a median 20 ms and runtime boot a further 15 ms, nearly identical across servers because neither depends on the application. We cross-checked launch in isolation: spawning `/usr/bin/true` takes 1.2 ms, `node -version`—which exits before V8 starts—12.9 ms, and `node -e 0` 21.4 ms end to end. The floor of process launch is therefore loading the 68 MB `node` binary and its libraries; within the benchmark loop it varies between 12 and 27 ms from run to run. For the bare `node:http` server this fixed cost is the whole cold start.

Finding 2. Initialization, not the runtime, separates slow servers from fast ones, and it grows with the number of modules loaded.

Module loading and application initialization take a median 50% of the cold start, and 80% for the servers slower than 150 ms. Fig. 3 plots initialization time against the number of modules each server loads before listening. The relationship is strong on a log–log scale ($r = 0.72$) and sublinear (slope 0.51), at a median 0.27 ms per module. The self-hosted applications are the heaviest: Verdaccio and Actual Budget each load over a thousand modules.

Finding 3. The module system, not dependency or application code, dominates initialization.

Fig. 4 attributes the CPU samples taken between the first user code and `listen()`. Each sample is charged to the innermost frame that belongs to the module loader, to a dependency, or to the application, so that work done on the loader’s behalf—file reads and URL handling during resolution, compilation called from the loader—counts as module loading. Module loading takes a median 72% of initialization, and at least 56% for each of the 8 servers with more than 50 ms of it; the dependencies’ own top-level code takes a median 23% there, and application code at most 4%. Counting only samples inside the loader’s own frames, the module system is still the largest

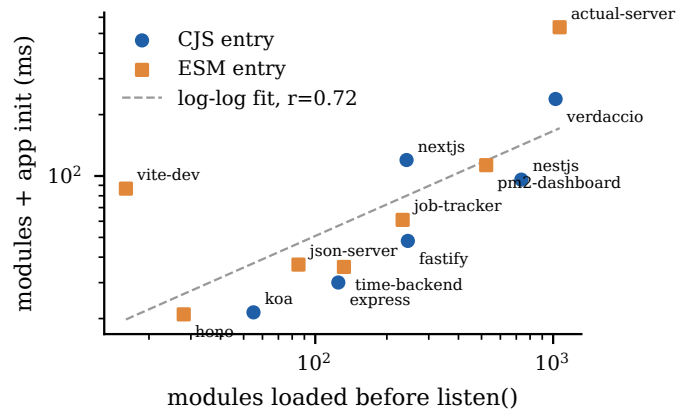


Fig. 3. Initialization time against modules loaded before `listen()` (servers with more than 10 modules).

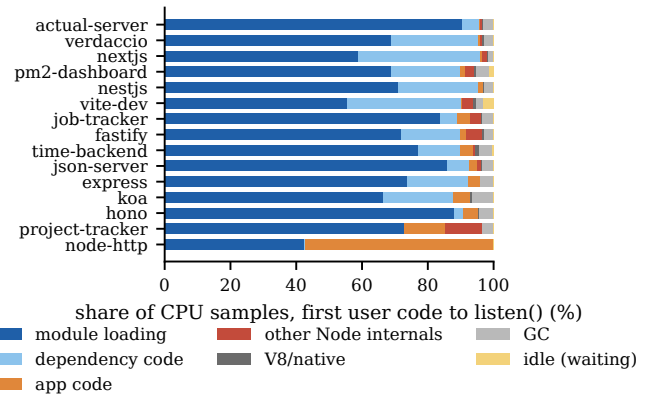


Fig. 4. What initialization is spent on: CPU samples (100 μ s) from the first user code to `listen()`, median of five profiled runs, each charged to the innermost module-loader, dependency or application frame. The `node:http` server’s initialization lasts only ≈ 2 ms.

single category (median 36% for the same servers). No server spends more than 3.3% of initialization idle: none of these cold starts waits on I/O.

Finding 4. Listening is not the end of a cold start for server-rendering frameworks.

For most servers the first request completes in a few milliseconds (median 3.8 ms against 0.4 ms warm). Next.js is the exception: it loads and compiles its rendering path lazily, so its first request takes 83 ms against 2.7 ms warm, over a third of its cold start. The Vite development server keeps working after it listens (it pre-bundles dependencies in the background), so its warm requests are slow for the first second or so.

V. WHAT DOES THE LAUNCHER ADD?

Next we add back what the ideal launcher removed. A request “arrives” for a stopped server and we time until its response completes, starting the server with `pm2` [5] or a direct spawn, and detecting readiness by TCP polling every 150 ms (the gateway’s original default) or 10 ms, by the probe’s `listen` notification, or not at all: with *socket pre-binding* the launcher binds the port itself, passes the listening socket to the server,

TABLE II
LAUNCHER LATENCY (MS), REQUEST ARRIVAL TO COMPLETE RESPONSE,
MEDIAN OF 20. BOLD: FASTEST.

App	pm2			direct spawn			
	poll 150	poll 10	notify	poll 150	poll 10	notify	prebind
node-http	160	85	80	159	49	45	45
express	160	108	97	158	72	69	70
fastify	159	120	116	157	92	89	88
koa	167	105	97	166	72	63	68
hono	160	103	93	159	64	61	61
nextjs	314	187	176	157	147	130	139
nextjs	272	275	268	229	234	236	241
json-server	160	115	113	158	88	74	73
verdaccio	464	321	311	310	284	284	275
actual-server	772	633	635	614	595	585	586
time-backend	162	114	106	160	88	81	74
project-tracker	160	92	97	158	65	65	63
job-tracker	159	127	128	157	109	102	92
pm2-dashboard	315	193	183	157	160	151	146
vite-dev	244	235	161	204	196	126	128

and forwards the request at once, so it waits in the kernel’s accept queue until the server accepts it. Pre-binding needs no application changes: the probe redirects the server’s own `listen()` call to the inherited socket. Table II gives the medians.

Finding 5. For small servers, the launcher costs more than the server.

The gateway as shipped (pm2 with 150 ms polling) is slower than the best launcher by a median 54% of its latency, and by up to 72%. Most personal servers are small: for the servers that start in under 100 ms, the launcher accounts for 58% of what the user waits.

Finding 6. Polling quantizes latency.

With a 150 ms interval, every server that starts in under 150 ms is answered at ≈ 160 ms, and one that needs slightly more waits for the second poll (NestJS and the pm2 dashboard under pm2: ≈ 313 ms). The median penalty over a `listen` notification is 66 ms. A 10 ms interval recovers almost all of it (median 7.7 ms); notification removes the rest and the polling traffic. Probing is not always free: in a controlled test, TCP probes that continue while the Vite development server handles its first request raise that request from 10 to 67 ms, because the server does work for every connection it accepts; probes that stop as soon as the port accepts cost nothing measurable.

Finding 7. pm2 adds a constant ≈ 32 ms, and it is spent inside the application process.

Under a `listen` notification, pm2 is slower than a direct spawn by a median 32 ms (range 25–49). Its daemon is not the cost: the application’s first user code runs at the same time under pm2 as under a direct spawn (median difference -2 ms). The overhead lies between that point and `listen()` (median 38 ms), where pm2’s fork wrapper runs before the application, and it traces to one default (§V-A).

Finding 8. Socket pre-binding adds nothing over a `listen` notification.

Pre-binding and notification are within noise of each other (median difference 0.1 ms): once the launcher learns of `listen()`

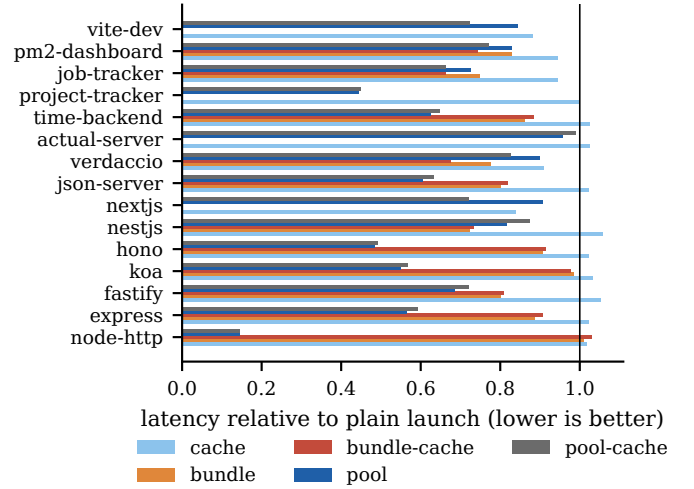


Fig. 5. Latency of each technique relative to a plain launch (median of 20).

within a millisecond, holding the request in the kernel’s queue instead of in the launcher saves nothing. Its remaining value is robustness—no window in which connections are refused—not speed.

A. One default: pm2’s monitoring agent

pm2’s fork-mode wrapper calls `require('@pm2/io')` before loading the application, unless the process is started with `pmx: false`. The agent provides PM2.io metrics that a local gateway does not use. Turning it off (E2b: 20 repetitions per server, interleaved with pm2 with the agent and a direct spawn) saves a median 24 ms per start (range 2–34 ms; clear for 15 of 15 servers) and leaves pm2 a median 7.9 ms slower than a direct spawn.

VI. APPLICATION-TRANSPARENT TECHNIQUES

We now evaluate techniques a launcher can apply to a server it did not write, each measured from launch to the first complete response with `listen` notification (Table III, Fig. 5). *Compile cache* sets `NODE_COMPILE_CACHE` to a per-server directory [16]; we report the run that fills it and the steady state. *Bundle* replaces the entry with an esbuild [19] single-file bundle placed beside it, so that paths relative to the entry still resolve; native add-ons and unresolvable optional dependencies stay external. *Pool* keeps a pre-started, application-agnostic Node.js process that, on demand, switches to the server’s directory, environment and arguments and runs its entry through Node’s normal main-module path. Each artifact was checked to return the same first response as the unmodified server.

Finding 9. V8’s compile cache barely helps servers.

In steady state the cache changes latency by a geometric mean factor of 0.98. It helps only the servers with the most compiled code—Next.js, the Vite server, Verdaccio, the job tracker and the pm2 dashboard gain 6–16%—and it makes smaller servers up to 6% slower, as reading cache entries costs more than the compilation it replaces; the run that fills the cache is up to 8% slower. It does nothing for Actual Budget, the slowest

TABLE III

LATENCY RELATIVE TO A PLAIN LAUNCH (MEDIAN OF 20; PLAIN LAUNCH IN MS). BOLD: CLEARLY FASTER, ITALIC: CLEARLY SLOWER (95% BOOTSTRAP CI OF THE DIFFERENCE EXCLUDES ZERO). –: TECHNIQUE NOT APPLICABLE.

App	base (ms)	cache	cache (first)	bundle	bundle+cache	pool	pool+cache
node-http	41	1.02	1.02	1.01	1.03	0.15	0.15
express	65	1.02	<i>1.03</i>	0.89	0.91	0.56	0.59
fastify	80	1.05	<i>1.05</i>	0.80	0.81	0.69	0.72
koa	61	<i>1.03</i>	<i>1.03</i>	0.99	0.98	0.55	0.57
hono	57	1.02	1.03	0.91	0.91	0.48	0.49
nestjs	128	<i>1.06</i>	<i>1.08</i>	0.72	0.73	0.82	0.87
nextjs	229	0.84	<i>1.06</i>	–	–	0.91	0.72
json-server	70	<i>1.02</i>	<i>1.04</i>	0.80	0.82	0.61	0.63
verdaccio	280	0.91	<i>1.04</i>	0.78	0.68	0.90	0.83
actual-server	566	<i>1.03</i>	<i>1.03</i>	–	–	0.96	0.99
time-backend	70	<i>1.02</i>	<i>1.01</i>	0.86	0.88	0.63	0.65
project-tracker	55	1.00	1.04	–	–	0.44	0.45
job-tracker	94	0.94	<i>1.05</i>	0.75	0.66	0.73	0.66
pm2-dashboard	144	0.94	<i>1.03</i>	0.83	0.74	0.83	0.77
vite-dev	123	0.88	<i>1.05</i>	–	–	0.84	0.72
geo. mean		0.98	1.04	0.84	0.82	0.62	0.61

server, consistent with Finding 3: its initialization is spent in the module system, not in compilation.

Finding 10. Bundling pays off where it works, but works for only 11 of 15 servers.

Where the bundle is correct, it lowers latency by a geometric mean factor of 0.84 (up to 28% for NestJS), because a bundle turns hundreds of module resolutions and file reads into one. Combined with the compile cache it reaches 0.66–0.68 for the job tracker and Verdaccio. The four failures show three ways in which bundling changes semantics: Next.js cannot be bundled at all; Vite and Actual Budget locate files (Vite’s `package.json`, Actual’s database migrations) relative to their own source files, which move when bundled; and the project tracker’s “am I the main module?” check becomes true inside the bundle, so it starts its server twice.

Finding 11. User-land startup snapshots are infeasible for every server.

We built snapshots from the CommonJS bundle, deferring `listen()` during the build and replaying it on the runtime port after deserialization. No server could be snapshotted. The six servers whose builds reached serialization all failed on the same object: `node:http`’s HTTP parser binding holds native handles that V8 cannot serialize, and Node.js warns that the module is not yet supported in snapshot builder scripts [17]. The others failed earlier: four use top-level `await` and three `import.meta`, neither of which survives the conversion to the CommonJS form snapshots require; Verdaccio resolves a logging transport module by name during initialization, which the snapshot builder, restricted to built-in modules, cannot do; and Next.js cannot be bundled. Because every web server loads `node:http` during initialization, snapshots cannot help this class of program until the module becomes snapshot-safe. The potential is large: a snapshot could remove the initialization phase, a median 50% of a cold start.

Finding 12. A pre-started runtime removes most of the fixed cost, for 42 MB.

TABLE IV

FREEZING: FIRST-RESPONSE LATENCY AFTER SIGCONT (MEDIAN OF 10) AND RESIDENT MEMORY HELD WHILE FROZEN, AGAINST A COLD START.

App	cold start (ms)	resume (ms)	RSS kept (MB)
node-http	45	1.8	45
express	70	1.8	59
fastify	87	1.7	65
koa	67	1.8	62
hono	60	1.6	58
nestjs	135	1.9	79
nextjs	237	9.8	121
json-server	75	1.8	64
verdaccio	277	2.5	139
actual-server	576	2.4	236
time-backend	76	3.8	66
project-tracker	58	2.5	58
job-tracker	100	2.3	75
pm2-dashboard	152	3.0	89
vite-dev	132	3.0	245

The pool lowers latency by a geometric mean factor of 0.62, saving a median 26 ms, most of Node’s fixed launch and boot cost (Finding 1); handing the application to the waiting process costs the rest. For the bare `node:http` server, cold start falls from 41 to 6 ms. One idle pooled process holds 42 MB, and one process can serve any server, so a single spare process covers the common case of one cold start at a time.

A. Freezing instead of stopping

Instead of stopping an idle server, a gateway can freeze its process group (SIGSTOP) and resume it (SIGCONT) on the next request. The server’s listening socket stays open while it is frozen, so a request arriving during the resume is queued by the kernel.

Finding 13. Freezing resumes in 2.3 ms but keeps all of its memory.

A frozen server answers its first request a median 2.3 ms after resumption (at most 9.8 ms, for Next.js), one to two orders of magnitude faster than a cold start (Table IV). But on an

TABLE V

END TO END THROUGH THE GATEWAY: REQUEST TO A STOPPED SERVICE UNTIL ITS RESPONSE (MS, MEDIAN OF 20). COLUMNS AFTER “POLL 150” ARE CUMULATIVE.

Service	poll 150	poll 10	notify	+ no agent	+ fast path
job tracker	252	197	197	182	160
pm2 dashboard	263	255	252	239	186
project tracker	251	167	161	145	102
time tracker (API)	255	187	177	159	115
time tracker (Vite)	403	309	279	284	221

unpressured machine, macOS reclaimed none of its resident memory after five seconds: a frozen server holds 100% of its warm footprint, a median 66 MB and up to 245 MB for the Vite server. Freezing all 15 servers would hold 1459 MB. Freezing is thus a latency tier between warm and cold that saves CPU, not memory, unless the OS is under pressure and compresses or swaps the frozen pages—the regime Hibernate Container forces deliberately [20].

VII. END TO END: AN ON-DEMAND GATEWAY

We applied the findings to LOCALSERVER, the gateway that motivated this study, without touching any application. Four changes are settings: (1) readiness by *listen notification*—the gateway injects a short probe into every managed service through `NODE_OPTIONS`, so it also reaches Node.js processes started through `npm` or other wrappers, and the probe reports `listen()` over a Unix socket, with 10 ms polling as a fallback for other runtimes; (2) `pm2` started with `pmx: false`; (3) an optional per-service compile cache; and (4) a *fast control path*, described below. The pool and bundling are not yet integrated: `pm2` cannot adopt a process it did not fork, and bundling needs a per-application correctness check. We measured the five services of the four personal applications end to end through the gateway: stop the service, send a request to its domain, and time the response, 20 times per configuration (Table V). For the Vite server the probe request is not an HTML navigation, which Vite answers with 404 once it is up; the latency still includes its full start.

Finding 14. The gateway’s own control path cost as much as starting a small server.

Instrumenting the gateway’s start path showed that before starting a service it asked `pm2` to describe the process (median 48 ms, over 316 starts), deleted `pm2`’s stored definition (24 ms) and created it again (19 ms)—round trips to `pm2`’s daemon that together exceed the whole cold start of the smallest servers. The describe call exists only to adopt a process that is already running, which the gateway’s periodic synchronization with `pm2` already detects, and re-creation is only needed when the service’s definition has changed. The fast control path uses the cached process list and, when a fingerprint of the definition is unchanged, starts the stopped process in place through the daemon’s `startProcessId` operation (3 ms), falling back to the full path if the cached process has vanished. This alone saves a median 46 ms per cold start (25–54 ms); this is the

same class of overhead as the control-path latency reported for production serverless platforms [11].

Finding 15. Together, application-transparent changes cut the gateway’s cold-start latency by a median 45%.

Notification, disabling `pm2`’s agent and the fast control path reduce the latency a user waits for a stopped service by 29–59%, a median 140 ms; for the project tracker, from 251 to 102 ms. The compile cache added nothing measurable here (Table V omits it; median improvement with it 29% versus 30% without, before the control-path change), consistent with Finding 9.

VIII. DISCUSSION

For gateway and tool builders. The cheapest wins are outside the application. Notify on `listen()` instead of polling, or poll at 10 ms; if a process manager is used, disable features the gateway does not need (`pm2`’s agent costs ≈ 24 ms per start); and keep one pre-started runtime to absorb most of the fixed 35 ms. Treat the control path as part of the cold start: every round trip to a process manager’s daemon is paid on the critical path, and caching what the gateway already knows removed 46 ms per start. Enable the compile cache only for servers with large compiled graphs (Next.js, dev servers), and bundle only after checking the bundle’s first response matches, since bundling fails silently in ways that change behaviour. Freeze instead of stopping when latency matters more than memory.

For the Node.js runtime. Two findings point at the runtime rather than at applications. First, the module system—resolution, file access and linking—is the largest single cost of initializing a server, and grows with the number of modules; a resolution cache or a persisted module graph would help every server without changes, as bundling shows, where bundling is possible. Second, user-land startup snapshots, the one mechanism that could skip initialization altogether, are unavailable to every HTTP server because `node:http` is not snapshot-safe. Making it so would open snapshots to the whole class of programs this paper studies.

For application authors. Servers pay for every module loaded before `listen()`. Deferring optional subsystems (the pattern FaaSLight and SlimStart automate for functions [13], [14]) and avoiding source-relative file lookups (which also block bundling) both shorten cold starts.

A. Threats to validity

One machine, one runtime. All measurements come from one Apple M4 Pro under macOS 26.6 and Node.js 25.2. Absolute numbers will differ on other hardware and operating systems, whose process creation and dynamic loading costs differ; we expect the *relative* findings (launcher versus server, module system versus code, technique ratios) to transfer, but have not verified this. **Warm page cache.** Every run found the application’s files in the page cache, as they are for a service restarted during a working session; after a reboot, cold-cache starts will be slower and bundling relatively more valuable. **Instrumentation.** The probe’s CommonJS wrapper

and listen hook run in every measured process; the module-counting hook and the profiler, which cost more, ran only in separate passes. Categorizing profile samples by call frame is approximate: native work is charged to the JavaScript frame that called it. **Corpus.** Fifteen servers cannot represent all Node.js applications. We chose framework baselines to isolate framework cost, real self-hosted servers, and our own applications, which motivated the study and may not be typical. **Process launch variance.** Process launch varied between 12 and 27 ms across runs of the same server; we report medians over 20–30 interleaved runs. **Two gateway sessions.** The fast control path was measured in a later session than the other gateway configurations, in blocks rather than interleaved, because the two control paths interact through pm2’s process table; its baseline was re-measured in the same session and agrees with the earlier measurement within 5 ms for four services and 17 ms for the Vite server.

IX. RELATED WORK

Platform-level cold starts. Serverless research has attacked cold starts mostly beneath the application: lean containers and pre-imported Python zygotes (SOCK [6]), initialization-less booting from checkpoints (Catalyzer [7]), microVMs (Firecracker [8]) and their snapshots, with working-set prefetching (REAP [9], FaaSnap [10]), CRIU-based prebaking [21], OS co-design for restore [22], and, in production, fork-based starts and seeded user code [11]. These mechanisms rely on containers, hypervisors or CRIU, none of which a macOS laptop offers, and most treat application initialization as a black box to be skipped. Our pre-started pool is the process-level analogue of the zygote idea; our snapshot results show why the runtime-level analogue does not yet work for Node.js servers. A survey organizes this space [2].

Keep-alive policy. Characterizations of production traces and caching-based policies decide *when* to keep instances warm [1], [12]. We study the cost paid when that decision goes the other way; the two are complementary, and our gateway now logs the usage data needed to evaluate such policies on a personal machine.

Application-level loading. FaaSLight separates optional from indispensable code in Python and JavaScript functions and loads the former on demand [13]; SlimStart and InitScope profile and refactor Python library loading and initialization anti-patterns [14], [15]. For server-side JavaScript, Stubbifier debloats Node.js applications with lazily-fetched stubs [23], Lazifier introduces lazy loading for front-end code [24], and Mininode removes unused code to reduce attack surface [25]. Kula et al. measured load times of individual npm packages [26]. These works change the application or its dependencies; we measure whole servers and evaluate what a launcher can do with them unchanged, and our module-system finding suggests where such transformations pay off.

Socket activation and freezing. inetd and systemd hand listening sockets to services they start on demand [27]; Node.js libraries and SvelteKit’s adapter support it when the application opts in [28], [29]. Our probe applies it to unmodified servers.

Android freezes cached apps rather than killing them [30], [31], and Hibernate Container deflates idle containers’ memory [20]; we quantify the plain freeze-versus-stop trade-off for local servers.

X. CONCLUSION

Scale-to-zero on a personal machine trades memory for cold starts that one user feels directly. For the Node.js servers we measured, those cold starts are a fixed 35 ms of runtime plus initialization dominated by the module system, and, for the small servers typical of personal use, the launcher around them costs more than the servers themselves. Polling, a process manager’s default agent and the runtime’s fixed cost are all removable without touching applications; the compile cache is not the lever it appears to be, and startup snapshots are closed to HTTP servers for now. A gateway applying these findings cut its cold-start latency by a median 45%. The companion question—when a gateway should stop, freeze or keep a server—needs usage data, which our gateway now records.

ARTIFACT

The harness, corpus manifest, raw measurements and analysis scripts are in the accompanying repository (`coldstart-study/`); `harness/run-all.sh` reproduces every experiment, and `harness/analyze.py` regenerates every table, figure and number in this paper.

REFERENCES

- [1] M. Shahrad, R. Fonseca, Í. Goiri, G. Chaudhry, P. Batum, J. Cooke, E. Laureano, C. Tresness, M. Russinovich, and R. Bianchini, “Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider,” in *Proc. USENIX Annual Technical Conference (ATC)*, 2020.
- [2] M. Golec, G. K. Walia, M. Kumar, F. Cuadrado, S. S. Gill, and S. Uhlig, “Cold start latency in serverless computing: A systematic review, taxonomy, and future directions,” *ACM Computing Surveys*, 2024.
- [3] E. Phoenix, “puma-dev: A tool to manage rack apps in development with puma,” <https://github.com/puma/puma-dev>.
- [4] Sablier contributors, “Sablier: Start your containers on demand, shut them down automatically when there is no activity,” <https://github.com/sablierapp/sablier>.
- [5] Keymetrics, “PM2: Production process manager for Node.js,” <https://pm2.keymetrics.io>, version 6.0.13.
- [6] E. Oakes, L. Yang, D. Zhou, K. Houck, T. Harter, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, “SOCK: Rapid task provisioning with serverless-optimized containers,” in *Proc. USENIX Annual Technical Conference (ATC)*, 2018.
- [7] D. Du, T. Yu, Y. Xia, B. Zang, G. Yan, C. Qin, Q. Wu, and H. Chen, “Catalyzer: Sub-millisecond startup for serverless computing with initialization-less booting,” in *Proc. 25th Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2020.
- [8] A. Agache, M. Brooker, A. Florescu, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, and D.-M. Popa, “Firecracker: Lightweight virtualization for serverless applications,” in *Proc. 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2020.
- [9] D. Ustiugov, P. Petrov, M. Kogias, E. Bugnion, and B. Grot, “Benchmarking, analysis, and optimization of serverless function snapshots,” in *Proc. 26th Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2021.
- [10] L. Ao, G. Porter, and G. M. Voelker, “FaaSnap: FaaS made fast using snapshot-based VMs,” in *Proc. 17th European Conference on Computer Systems (EuroSys)*, 2022.

- [11] X. Chai, T. Zhou, K. Hu, J. Tan, T. Bie, A. Shen, D. Shen, Q. Xing, S. Song, T. Yang, L. Gao, F. Yu, Z. He, D. Du, Y. Xia, K. Chen, and Y. Chen, "Fork in the road: Reflections and optimizations for cold start latency in production serverless systems," in *Proc. 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2025.
- [12] A. Fuerst and P. Sharma, "FaasCache: Keeping serverless computing alive with greedy-dual caching," in *Proc. 26th Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2021.
- [13] X. Liu, J. Wen, Z. Chen, D. Li, J. Chen, Y. Liu, H. Wang, and X. Jin, "FaaSLight: General application-level cold-start latency optimization for function-as-a-service in serverless computing," *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 5, 2023.
- [14] S. S. M. Tariq, A. A. Zein, S. S. Vaidya, A. Khanolkar, Z. Song, and P. Roy, "Efficient serverless cold start: Reducing library loading overhead by profile-guided optimization," in *Proc. 45th IEEE Int. Conf. on Distributed Computing Systems (ICDCS)*, 2025.
- [15] S. S. M. Tariq, F. Hassan, A. Bosu, and P. Roy, "Cold-start anti-patterns and refactorings in serverless systems: An empirical study," in *Proc. IEEE Int. Conf. on Software Analysis, Evolution and Reengineering (SANER)*, 2026, to appear; arXiv:2512.16066.
- [16] Node.js project, "Module compile cache (module.enableCompileCache, NODE_COMPILE_CACHE)," <https://nodejs.org/api/module.html>, node.js v25 documentation.
- [17] —, "Startup snapshot API (v8.startupsnapshot, -build-snapshot)," <https://nodejs.org/api/v8.html>, node.js v25 documentation.
- [18] Y. Guo, "Custom startup snapshots," V8 blog, <https://v8.dev/blog/custom-startup-snapshots>, 2015.
- [19] E. Wallace, "esbuild: An extremely fast bundler for the web," <https://esbuild.github.io>.
- [20] Y. Sun, D. Vij, F. Li, W. Guo, and Y. Xiong, "Hibernate container: A deflated container mode for fast startup and high-density deployment in serverless computing," arXiv:2305.10963, 2023.
- [21] P. Silva, D. Fireman, and T. E. Pereira, "Prebaking functions to warm the serverless cold start," in *Proc. 21st International Middleware Conference*, 2020.
- [22] B. Holmes, B. Dinis, L. Honcharuk, J. Fried, and A. Belay, "Taming serverless cold starts through OS co-design," arXiv:2509.14292, 2025.
- [23] A. Turcotte, E. Arteca, A. Mishra, S. Alimadadi, and F. Tip, "Stubbifier: Debloating dynamic server-side JavaScript applications," *Empirical Software Engineering*, 2022.
- [24] A. Turcotte, S. Gokhale, and F. Tip, "Increasing the responsiveness of web applications by introducing lazy loading," in *Proc. 38th IEEE/ACM Int. Conf. on Automated Software Engineering (ASE)*, 2023.
- [25] I. Koishybayev and A. Kapravelos, "Mininode: Reducing the attack surface of Node.js applications," in *Proc. 23rd Int. Symposium on Research in Attacks, Intrusions and Defenses (RAID)*, 2020.
- [26] R. G. Kula, A. Ouni, D. M. German, and K. Inoue, "On the impact of micro-packages: An empirical study of the npm JavaScript ecosystem," arXiv:1709.04638, 2017.
- [27] L. Poettering, "systemd for developers I: Socket activation," <https://0pointer.de/blog/projects/socket-activation.html>, 2011.
- [28] R. Vermeersch, "node-systemd: Support for running Node.js as a socket-activated service under systemd," <https://github.com/rubenv/node-systemd>.
- [29] SvelteKit, "Node servers: Socket activation," <https://svelte.dev/docs/kit/adapter-node>, accessed Sep. 2026.
- [30] Android Open Source Project, "Cached apps freezer," <https://source.android.com/docs/core/perf/cached-apps-freezer>, accessed Sep. 2026.
- [31] C. Li, Z. Zhu, C. J. Xue, Y. Liang, R. Ausavarungnirun, L. Shi, and X. Zhou, "Freezing-based memory and process co-design for user experience on resource-limited mobile devices," *ACM Transactions on Computer Systems*, 2025.